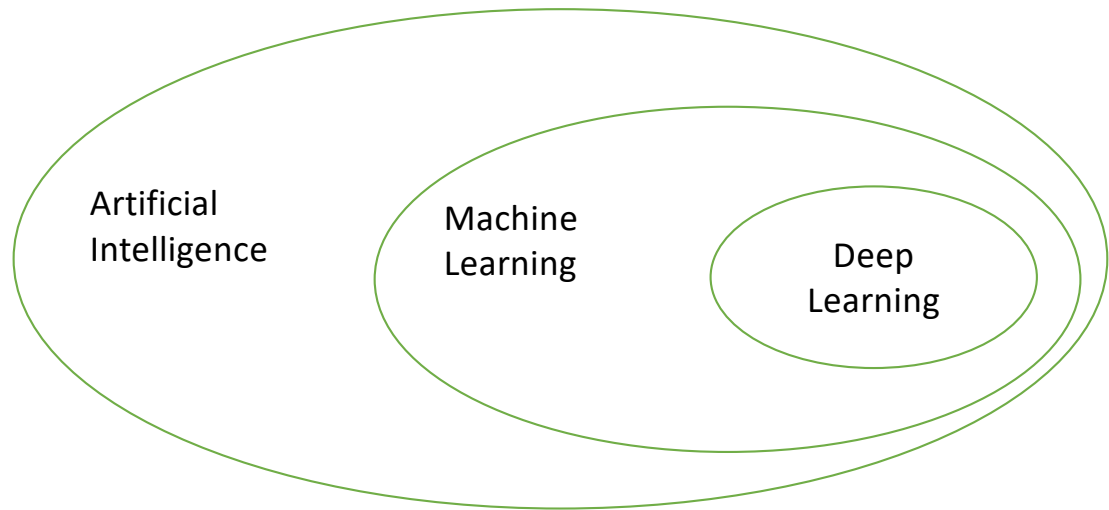


Deep neural networks

Deep learning: what is it?

Machine Learning field characterized by a greater complexity of models and the ability to learn representations from input data

Deep learning models consist of successive layers of representations, in a number that is typically high (deep models)



Deep learning: what is it?

Used models are typically based in neural networks structured in several processing layers

Different types of neurons and architectures used for different types of problems: feedforward neural networks, recurrent networks, convolutional networks, etc.

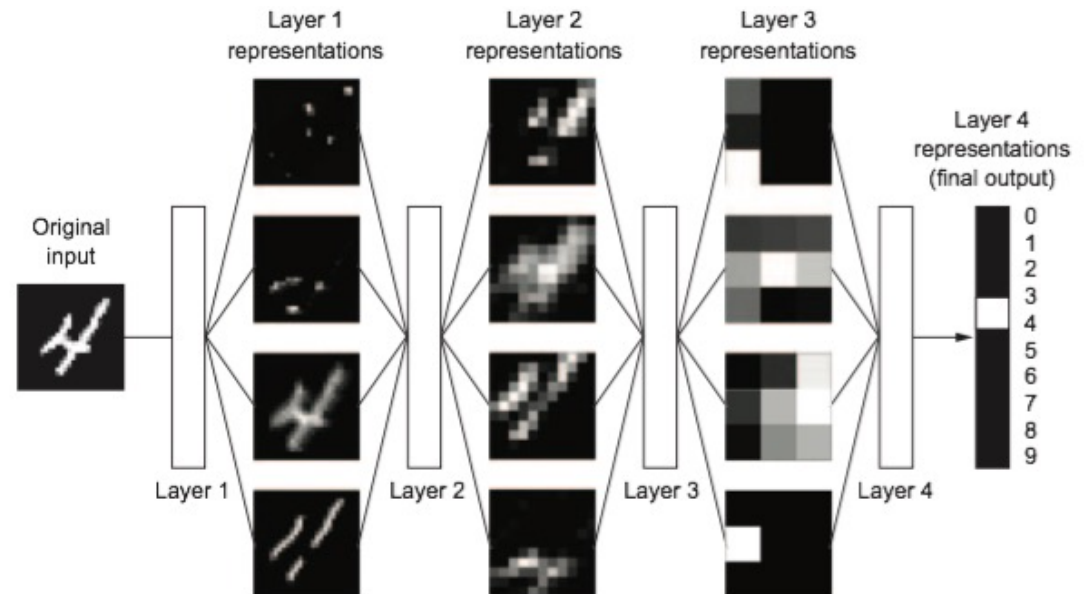
There are deep learning models for supervised and unsupervised learning problems, as well as reinforcement learning

Deep learning: what is it?

Several layers process information by creating distinct and typically more abstract representations of the inputs

In the case of supervised learning, the last layer represents the output of the neural net

Learning by **gradient descent** methods (next session)



Deep learning: major factors

Like any technology, DL does not solve all problems and will not always be the best option for any learning task

A determining factor for DL success is the availability of large-scale datasets; for problems with little data, other models can give more consistent results with less computational effort

In terms of hardware, the use of graphics processors (GPU) brings advantages in training DL models by accelerating the process by factors of more than 10x

Improvements in relation to "shallow" neural networks: activation functions (ReLU), weight initialization, optimization algorithms (RMSprop, Adam), methods for addressing overfitting, pre-training, and training "by layers"

Improvements on training algorithms: batches

Doing the training process using **mini-batches** (batches with part of the examples) instead of considering all available examples to calculate errors and their derivatives and update weights

Batch sizes to consider change typically between 64 and 512; *batch* should be of a size to allow keeping it as a whole in GPU memory

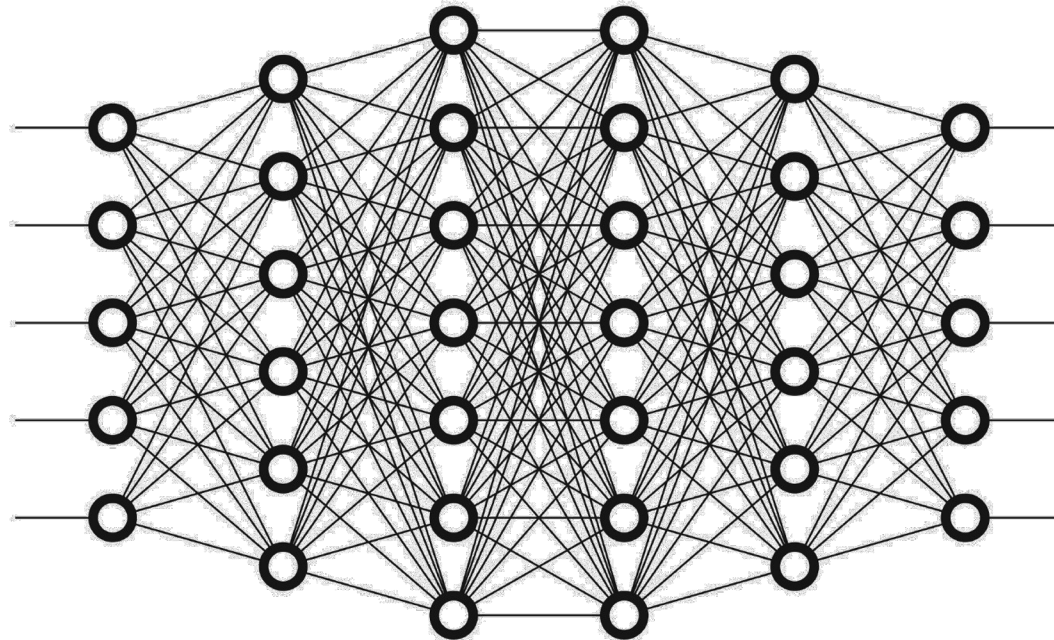
Batch size can be tuned/ optimized

If batch = 1 -> ***stochastic gradient descent***

If batch = number of examples in the dataset -> ***batch gradient descent***

Deep neural networks (DNNs)

DNNs are supervised DL models, being **feedforward** NNs with typically several hidden layers



Improvements on training algorithms: **momentum**

Recent algorithms use **momentum** – calculating moving average of the error gradients for the last N updates

Way to remove update noise, allowing to use higher learning rates and accelerate the convergence of the training process

Adds a new parameter (β) defining the weight to the previous value (new batch contributes with $1 - \beta$) – typical value of 0.9

Retained gradient $\rightarrow V_t = \beta V_{t-1} + (1 - \beta) \frac{\partial L}{\partial W}$

$$W_t = W_{t-1} - \alpha V_t$$

Improvements on training algorithms:

RMSProp e Adam

RMSProp also uses moving averages, but here of the squared errors (uses also a parameter β – typical value of 0.999)

Weight update divides the derivative by the square root of the moving average (adding a small value ϵ to avoid zero values)

Adam combines momentum with the strategy of RMSProp, so having two parameters for the update - β_1 e β_2 - and also ϵ

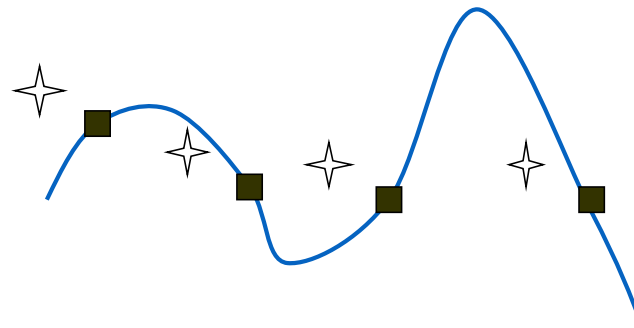
Overfitting and generalization in NNs

To train the network many epochs (training iterations) can prevent generalization and promote *overfitting*

It is preferable to stop the training in these cases – **early stopping**; error is measured in a validation dataset; if the validation error improves for a number of consecutive epochs, the training is stopped

The probability of overfitting increases if:

- Few training cases
- Many weights (complexity)



Overfitting and generalization

In DL, overfitting has been addressed using various techniques, such as **regularization** (L1 or L2, similar to linear/logistic regression, called **decay** in NNs), or explicit control of model **capacity** (e.g. number of hidden layers and number of neurons in each)

A specific technique developed for DL models is **dropout**: in each iteration of the training the output of some randomly selected neurons is put to zero. The proportion of zeroed neurons in each iteration is a parameter defined by each layer where dropout applies

Convolutional networks

What are convolutions

A convolution is a mathematical operation of two matrices:

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

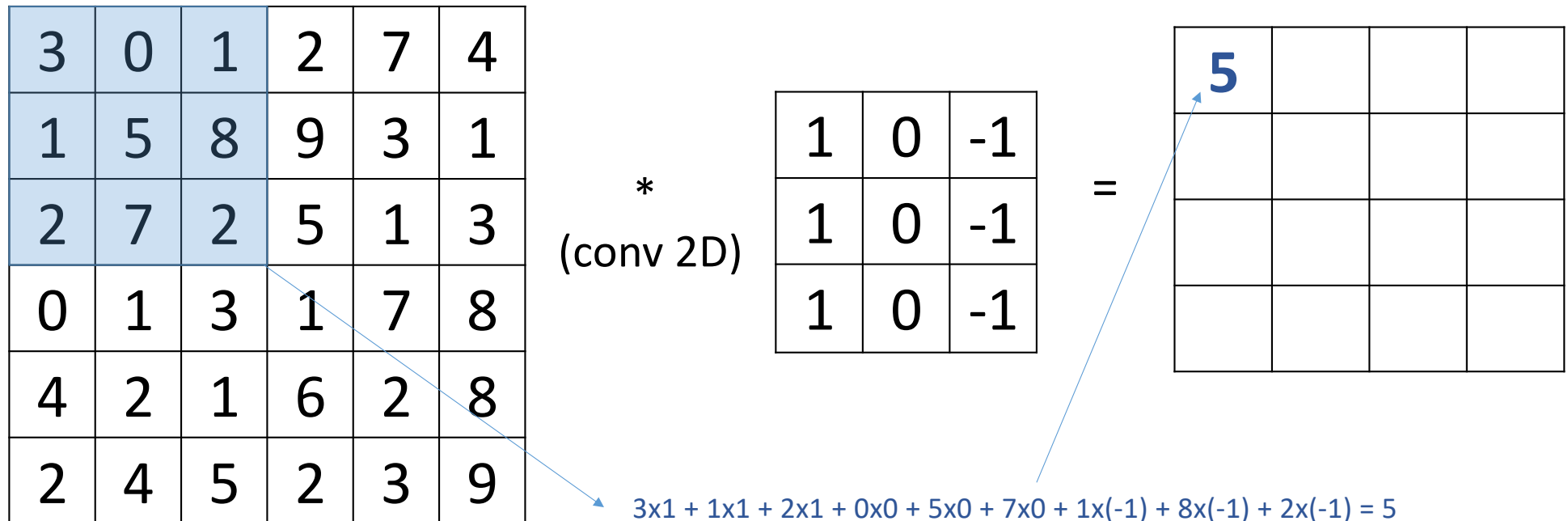
$*$
(conv 2D)

1	0	-1
1	0	-1
1	0	-1

=

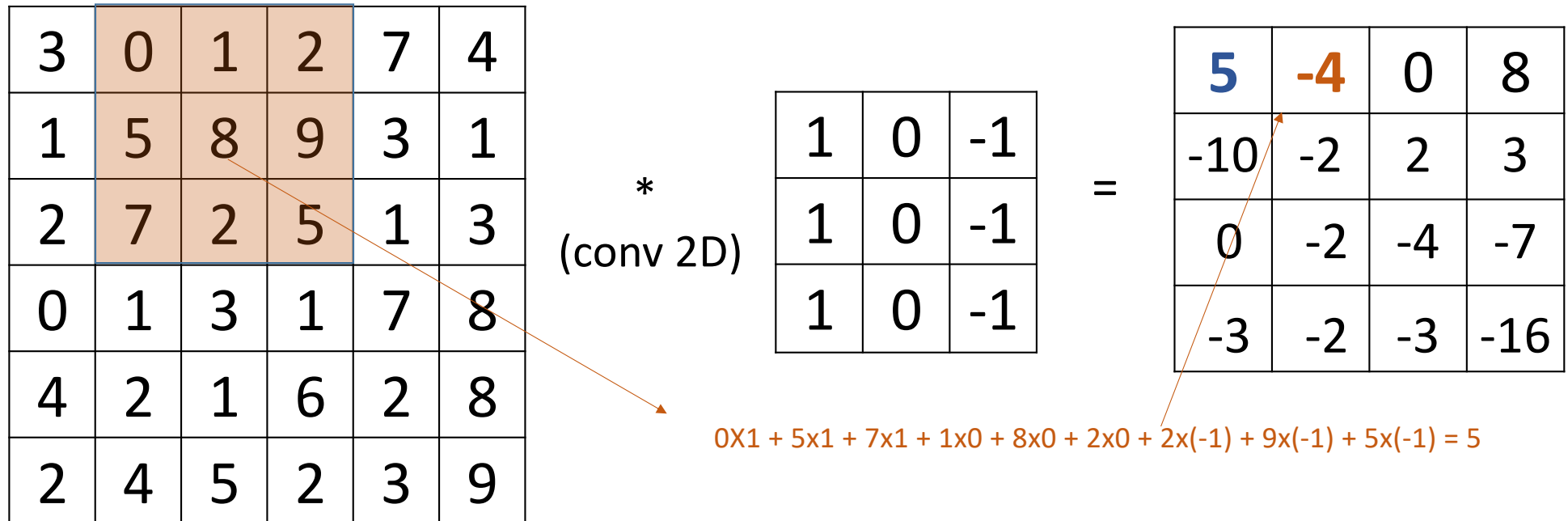
What are convolutions

Overlaps the 2nd matrix (filter) to the first in each position and multiplies the elements in each position, adding the result of the multiplications



What are convolutions

Overlaps the 2nd matrix (filter) to the first in each position and multiplies the elements in each position, adding the result of the multiplications



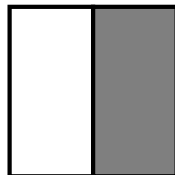
Convolutions as filters over images

In image processing, the convolutions, applied to various zones of the image, act as filters that can detect certain patterns

Examples are convolutions that can function as vertical or horizontal edges detectors

Convolutions as image filters: vertical edges

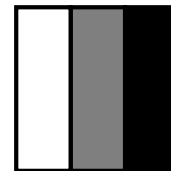
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

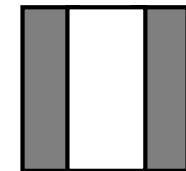
1	0	-1
1	0	-1
1	0	-1

*



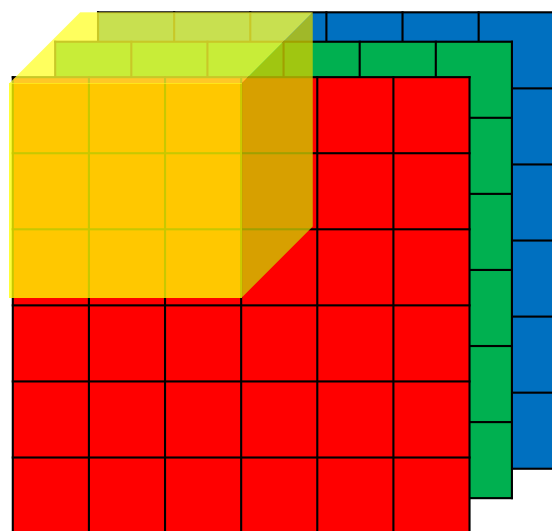
=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



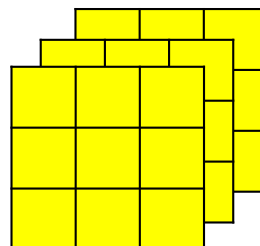
From Andrew Ng

Convolutions over 3d images



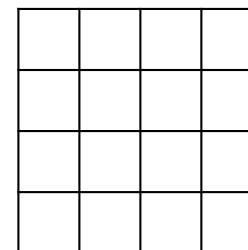
$6 \times 6 \times 3$

*



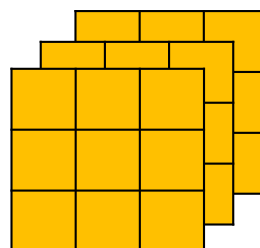
$3 \times 3 \times 3$

=



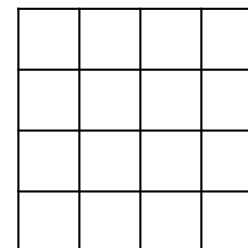
4×4

*



$3 \times 3 \times 3$

=



4×4

From Andrew Ng

Convolutional networks

CNN – Convolutional Neural Networks – are DL models where the various layers of the network are different from the feedforward ones, consisting in convolutional and “pooling” layers that receive inputs and generate outputs that are 3D tensors for each example

These are models typically applied to images where 3D tensors represent height, width and depth (or “channels”) – in the initial layers, these can be the dimensions of an image (e.g. RGB will have 3 channels); in the hidden layers, each channel works as a filter over the previous layer

These models represented a significant improvement in the way images are processed and classified

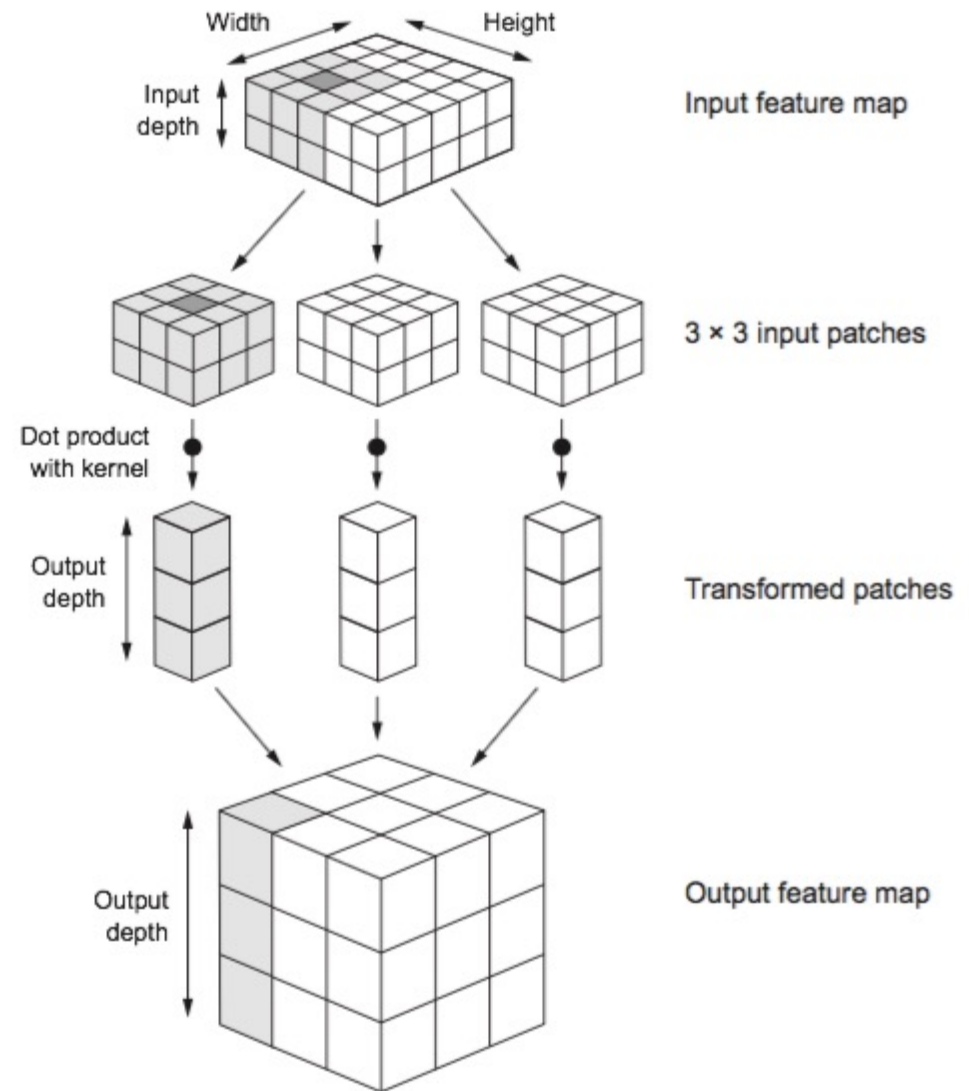
Convolutional layers

Work over “*feature maps*” – tensors 3D:
height x width x depth

All “patches” of a given dimension (e.g. 3x3 ou 5x5) are selected and a transformation is applied to each (e.g. tensors 3D 3x3xdepth)

Transformation: multiplication by the weights of “convolutional kernels” – these are learned via gradient descent (same kernel applied to all windows; number of kernels = output depth)

Transformation creates vector 1D of size equal to output depth



Pooling layers

Used to reduce the dimension of the output of conv layers through sampling

Extract Windows of the previous layer and calculates a single value for each window, typically considering the function ***max***

Another alternative can be the average (less used)

Typical values for the window are 2 x 2 or 3 x 3

These layers do not have any learnable parameter

They promote hierarchical learning

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2



9	2
6	3

Padding and stride

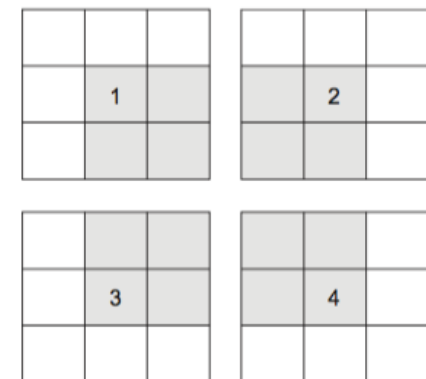
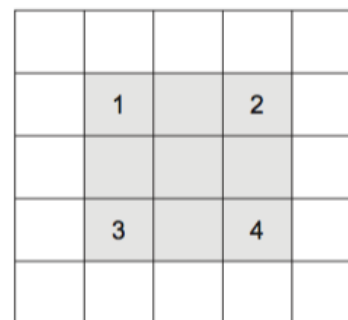
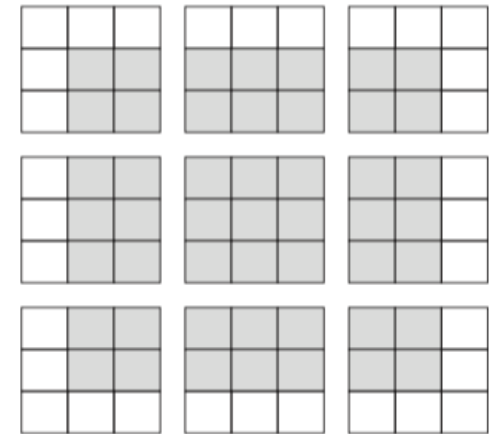
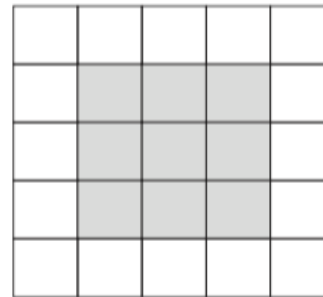
Dimension of conv layers (width, height) tend to get smaller given the valid positions to define windows

Padding can be used to maintain the same dimensions (consider extra rows and columns filled with zero)

The parameter *stride* controls the "steps" between convolutions

Example: stride = 2 ====>

Used as a default in max pooling layers

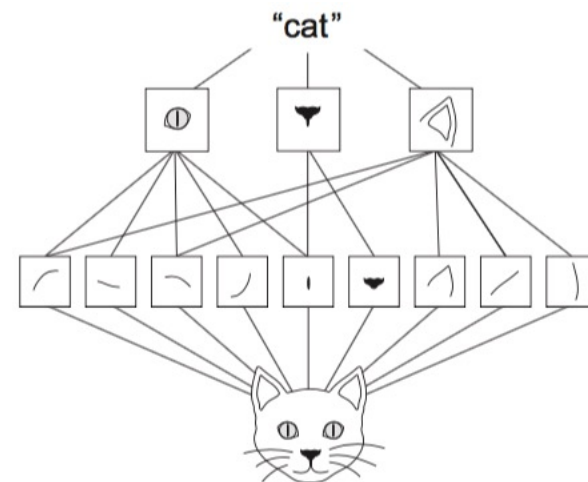
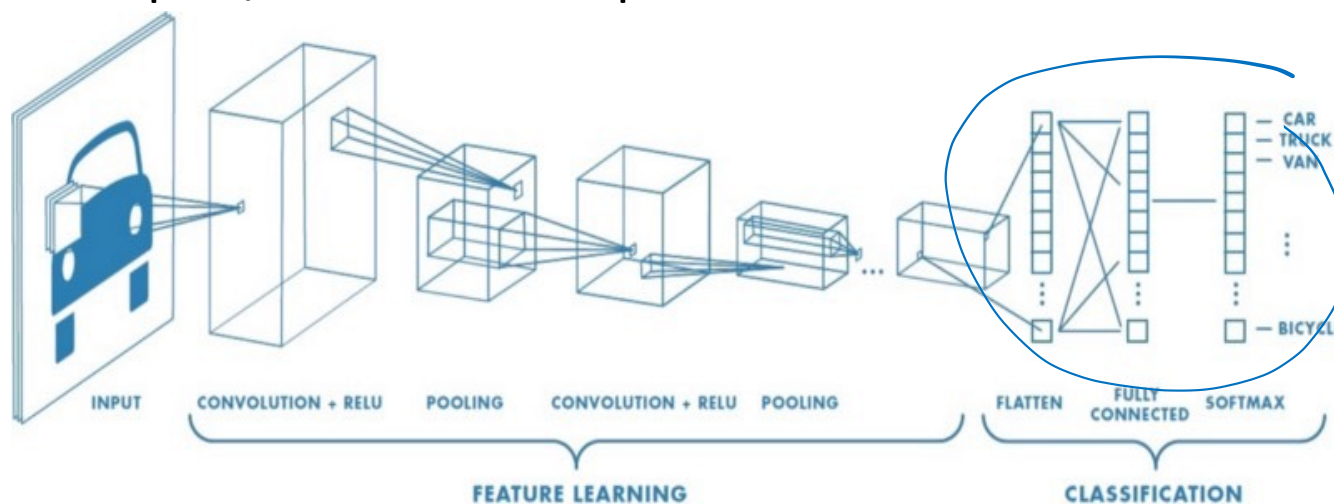


Learning in convolutional networks

Data processing in convolutional layers works over regions in the image, i.e. a matrix of pixels of small dimensions (e.g. 3x3)

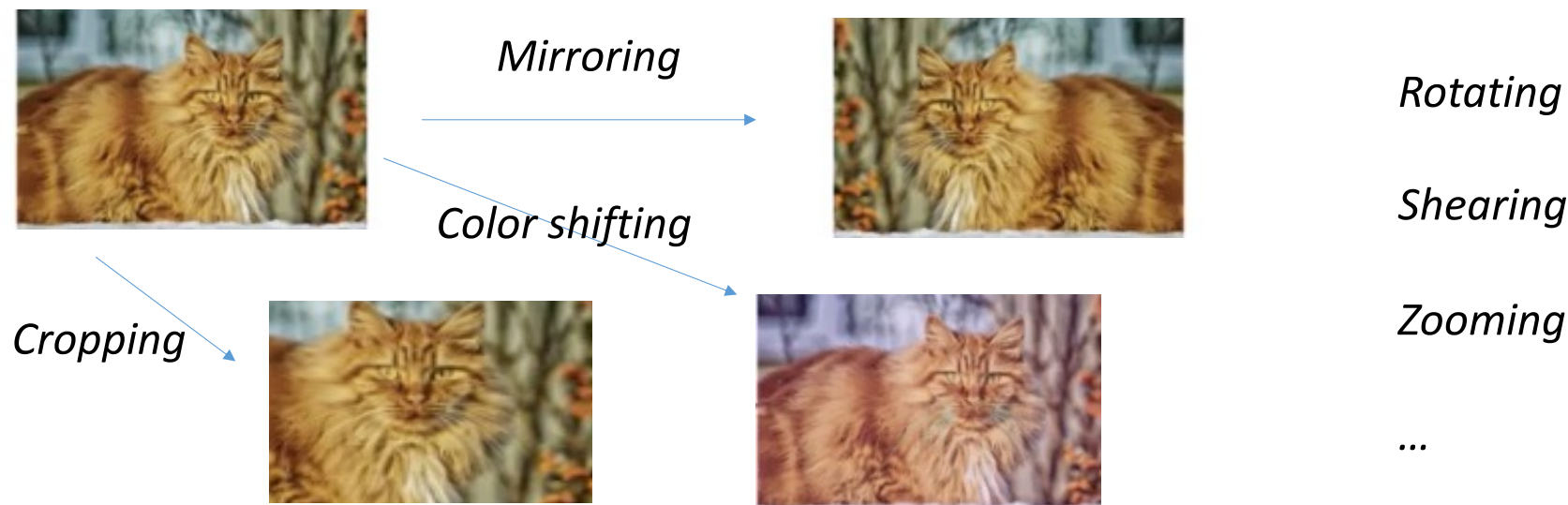
Allows to learn patterns that can be recognized in various parts of the matrix (e.g. image) – invariant to translations

Allows to learn hierarchically, with successive layers learning increasingly complex/abstract concepts



Extension of training data (*data augmentation*)

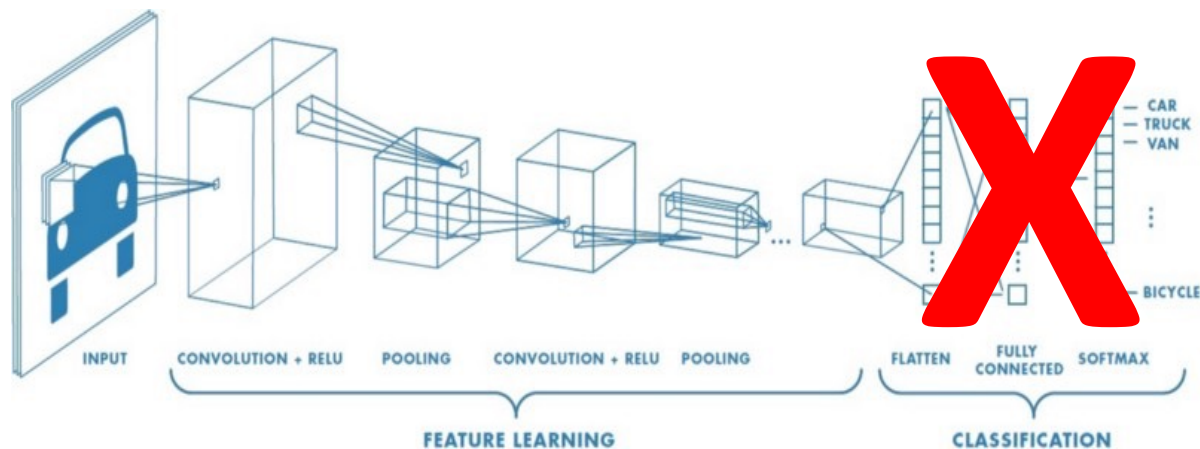
One of the solutions, when we have overfitting due to a small amount of data, is to create training examples based on existing ones, through image modifications that do not alter its essence, continuing to represent examples of the same class



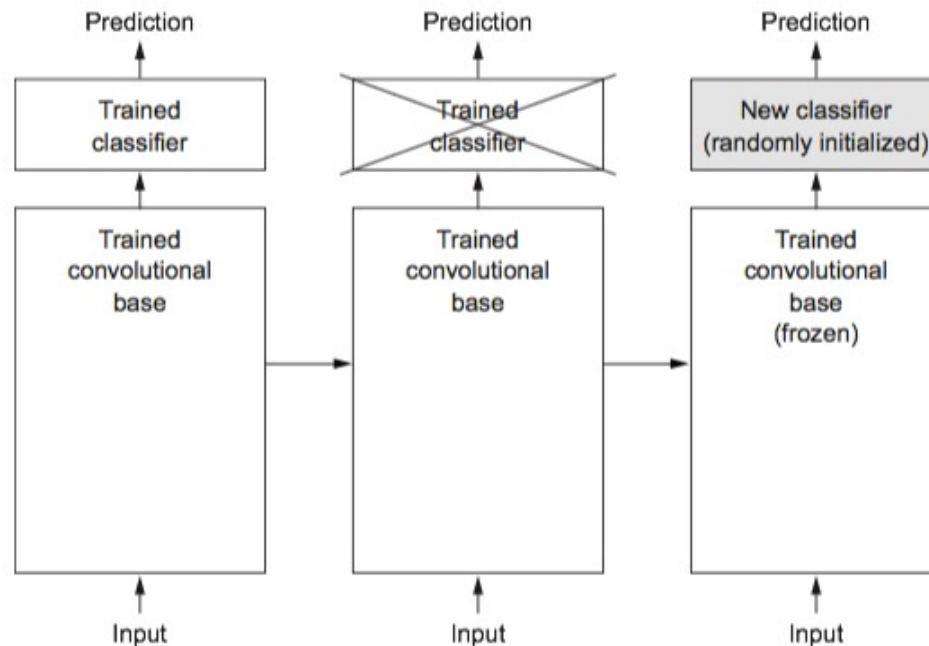
Using pre-trained models (*transfer learning*)

An alternative to improve models when we have little data goes through the use of pre-trained networks with larger datasets (created for other problems) and that can be refined with our dataset

Initial step: identify an appropriate existing network, import its topology and weights, remove parts that may not fit (e.g. classification part in fully connected layers)



Transfer learning: pre-training



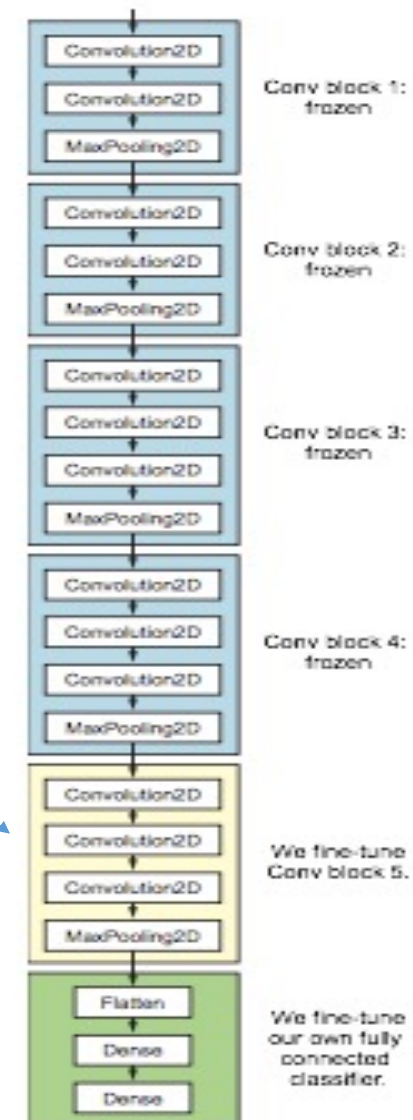
Convolutional base trained in the larger dataset and then maintained ("frozen" weights)

Use the convolutional base network with frozen weights, adding the prediction part with Dense layers, has advantages over simply using its outputs to train a DNN, because it allows to use data extension to prevent over-fitting

Transfer learning: fine-tuning

“Freeze” one or more layers
and re-train its weights

...



Handling sequence data

(mainly texts)

Processing sequence data

One area of success of DL is the ability to process sequential data, in particular textual and audio, as well as time series, with good performance

One of the most commonly used topologies in these applications are recurrent neural networks with various configurations

Applications include:

- NLP tasks (see next slide)
- Processing of audio and musical signals
- Time series forecasting
- Prediction of protein function, structure, and properties (in Bioinformatics)
- QSAR/ predicting activities of chemical compounds

Some tasks in Natural Language Processing (NLP)

- “What’s the topic of this text?” - **text classification**
- “Does this text contain abuse?” - **content filtering**
- “Does this text sound positive or negative?” - **sentiment analysis**
- “What should be the next word in this incomplete sentence?” - **language modeling**
- “How would you say this in German?” - **translation**
- “How would you summarize this article in one paragraph?” - **summarization**

Processing texts

DL models only process numerical data, and thus do not accept raw text – need to pre-process texts – **vectorization** – to convert into numerical vectors

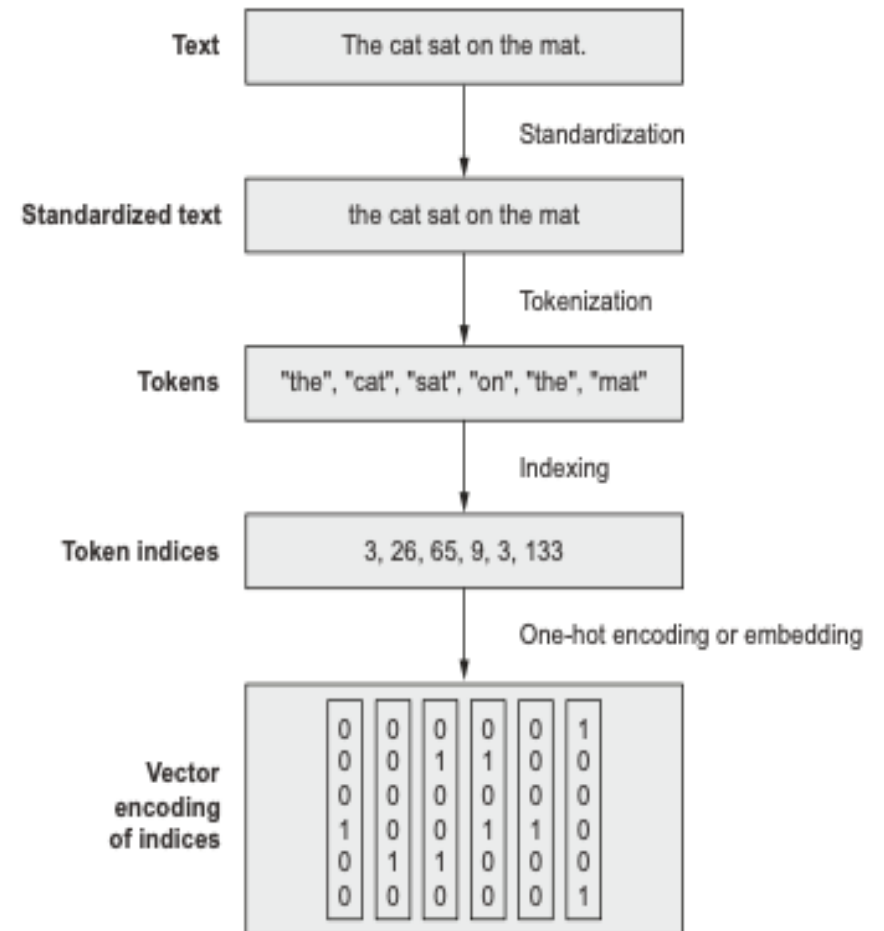
Several steps are usually taken:

- **Standardization** – removing punctuation; upper case to lower case; stemming; etc
- **Tokenization** - splitting the text into units (tokens), that may be words, groups of words, characters, groups of characters
- **Indexing** the tokens – converting into a numerical vector (e.g. one-hot encoding)

Processing texts

Typically, we see texts as word sequences

Texts are converted to numeric vectors, where words (or other types of tokens) are encoded as indexes of a vocabulary that typically has a predefined maximum size



Tokenization

- **Word-level tokenization**— tokens are space-separated (or punctuation-separated) substrings. A variant is to split words into subwords when applicable—for instance, treating “staring” as “star+ing” or “called” as “call+ed.”
- **N-gram tokenization**— tokens are groups of N consecutive words. For instance, “the cat” or “he was” would be 2-gram tokens (also called bigrams).
- **Character-level tokenization**— each character is its own token. In practice, this scheme is rarely used, only in specialized contexts, like text generation or speech recognition.

One-hot encoding: example

$V = [a, aaron, \dots, zulu, <UNK>]$

Vocabulary

Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$
5391	9853	4914	7157	456	6257

$|V| = 10000$

Most common representation:

One-hot encoding

where each word (token) has an index; representation of a word (token) is a vector with a value of 1 in that index and zeros in the remaining

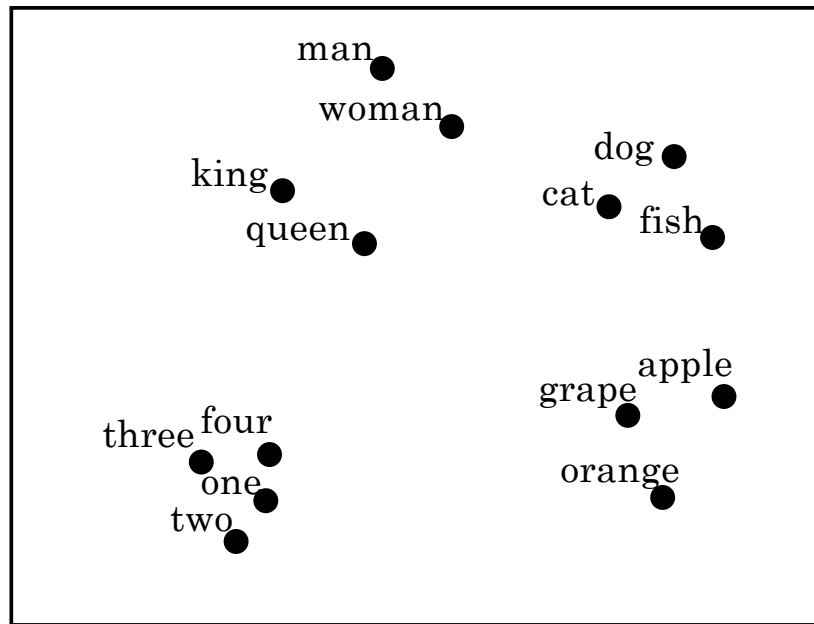
Usually, index 0 is for “mask” used for instance in padding
Index 1 is used for words not in vocabulary

Word embeddings

Each word is represented by a relatively small "dense" numeric vector (e.g. size 256-1024)

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
<i>Gender</i>	-1.0	1.0	-0.95	0.97	0.00	0.01
<i>Royal</i>	0.02	0.01	0.93	0.95	-0.01	0.00
<i>Age</i>	0.41	0.38	0.7	0.69	0.03	-0.02
<i>Food</i>	0.01	0.01	0.02	0.01	0.95	0.97

Visualizing word embeddings/ analogies



$$e_{man} - e_{woman} \approx e_{king} - e_{?}$$

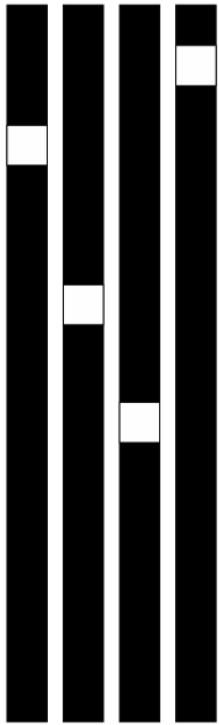
MAX:

$$\text{sim}(e_w, e_{king} - e_{man} + e_{woman})$$

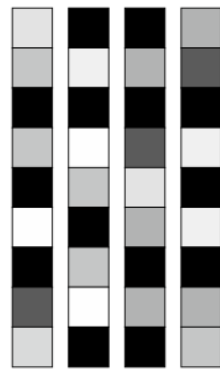
t-SNE

Allows to represent 2D embeddings
keeping distances

Word embeddings: characteristics



One-hot encoding



Embeddings

Word embeddings are

- Dense representations (vs sparse representations in one-hot encoding)
- Lower dimension
- Continuous values
- **Learned from data**

Learning embeddings

Embedding – matrix E :

- rows: dimensions of the embedding
- columns: words of the vocabulary

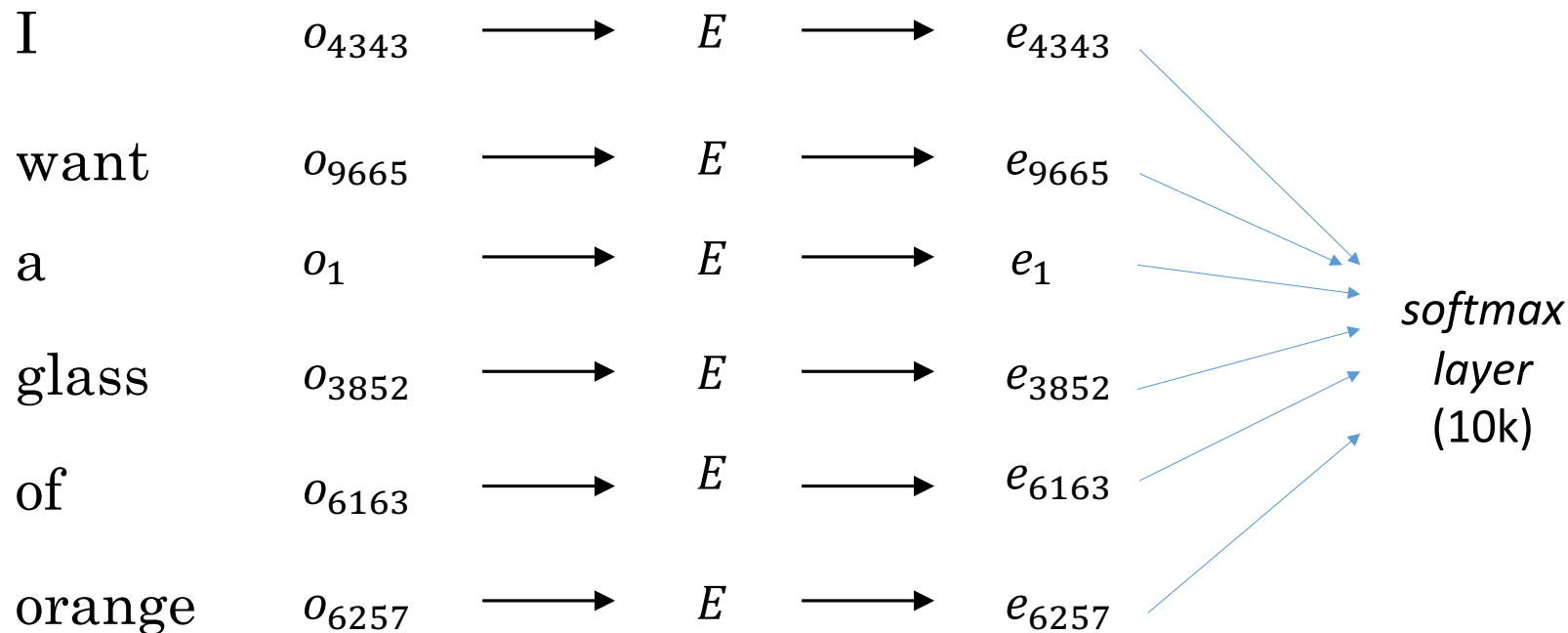
Note that by multiplying the representation of a word as *one-hot encoding* by E we have the column that corresponds to the embedding vector of that same word

Values of this matrix E can be learned as weights in any neural network by gradient descent, being matrix E the first layer, having as inputs one or more words and can have any type of task in subsequent layers

Learning embeddings: example

I want a glass of orange _____.

4343 9665 1 3852 6163 6257



What's the
next word ?

[Bengio et. al., 2003, A neural probabilistic language model]

Embeddings: transfer learning

Embeddings can be learned from scratch for each problem (if there is enough data)

In many cases, it is preferable to re-use pre-trained embeddings in extended datasets with large vocabularies
– *transfer learning*

If we have enough data, we can always refine embeddings with the available data

Embeddings most used: Word2Vec, Glove

Recurrent neural networks

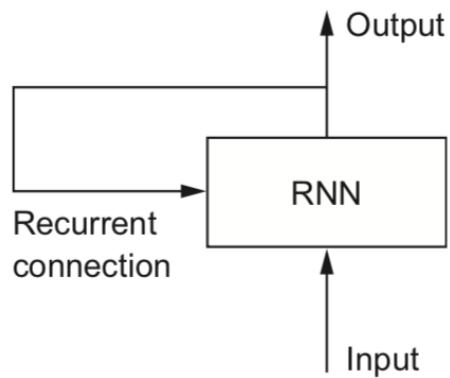
Unlike feedforward networks, recurrent networks have a topology that can contain cycles, with feedback from network outputs to inputs

Thus, they have the ability to process sequential data, and can maintain and can have various types of state / "memory" (of "short" and "long" term)

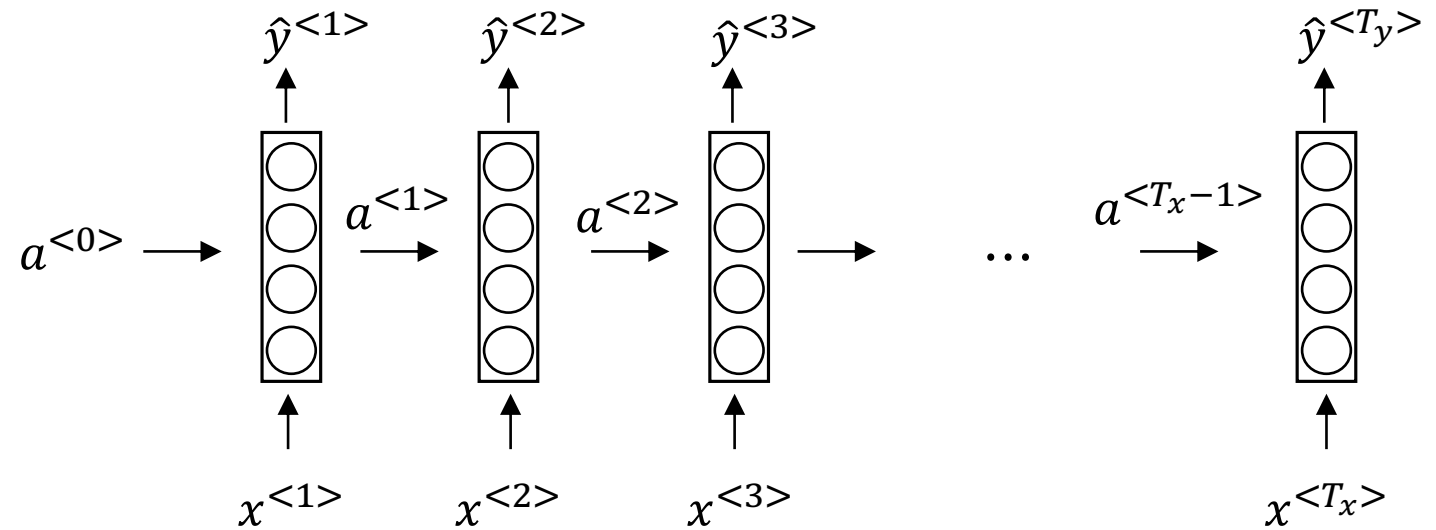
For example, in the case of IMDB, each review can be processed sequentially by being maintained a state relative to the words already seen (even if each review is analyzed independently)

Training algorithms based on gradient descent (e.g Backprop through time) propagate errors through the chain rule from the last state to the initial state

Recurrent neural networks

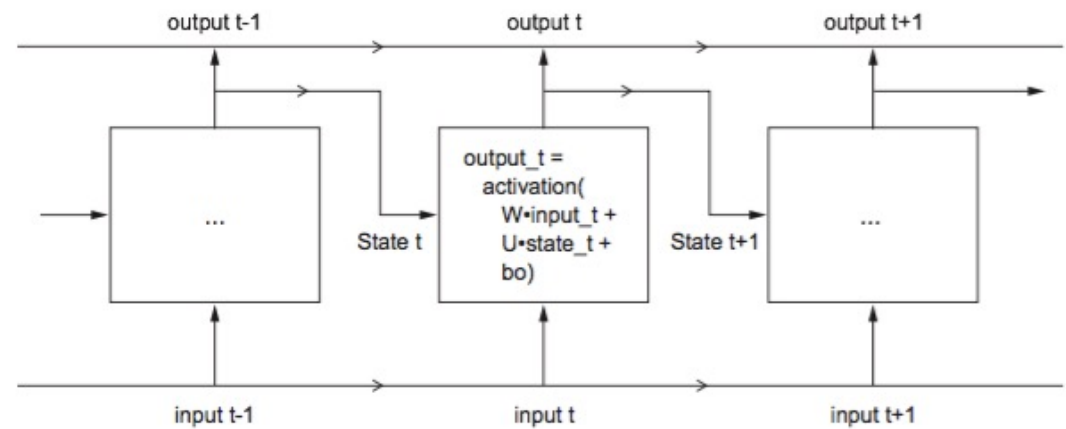


Example of a recurrent network
Layer ***SimpleRNN*** in keras

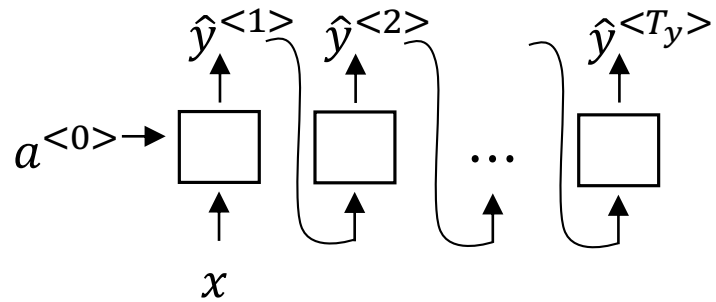


Calculation of outputs and states $\rightarrow$

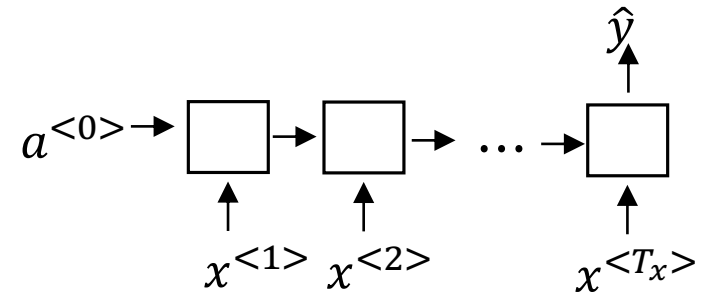
$\leftarrow$ Propagation of error



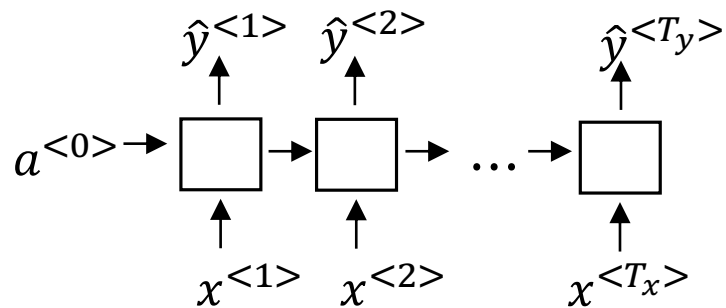
Several types of RNNs



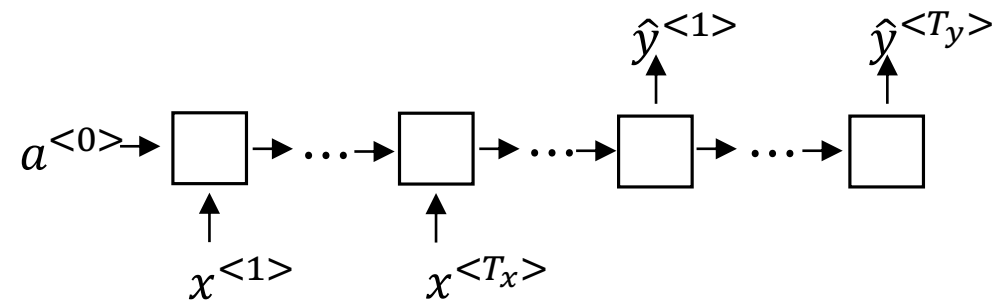
One to many



Many to one

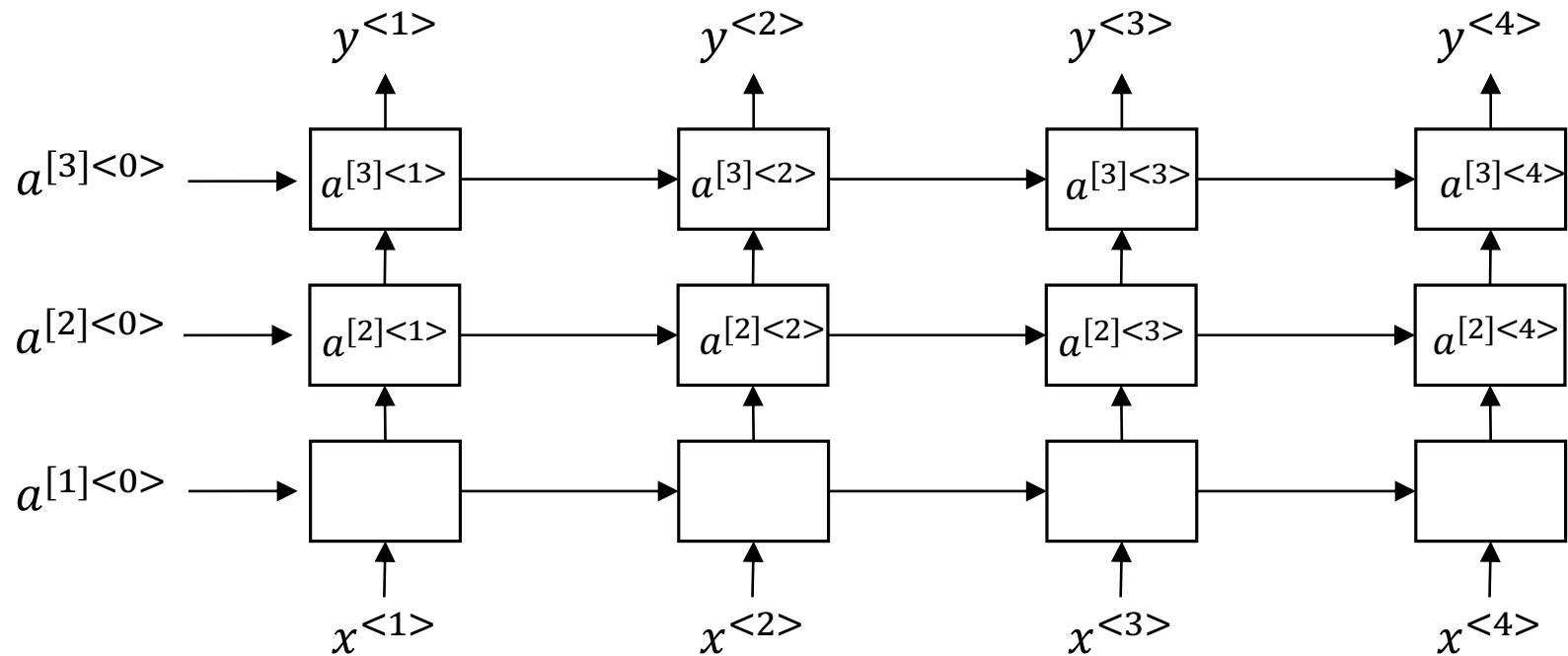


Many to many



Many to many

RNNs with multiple layers



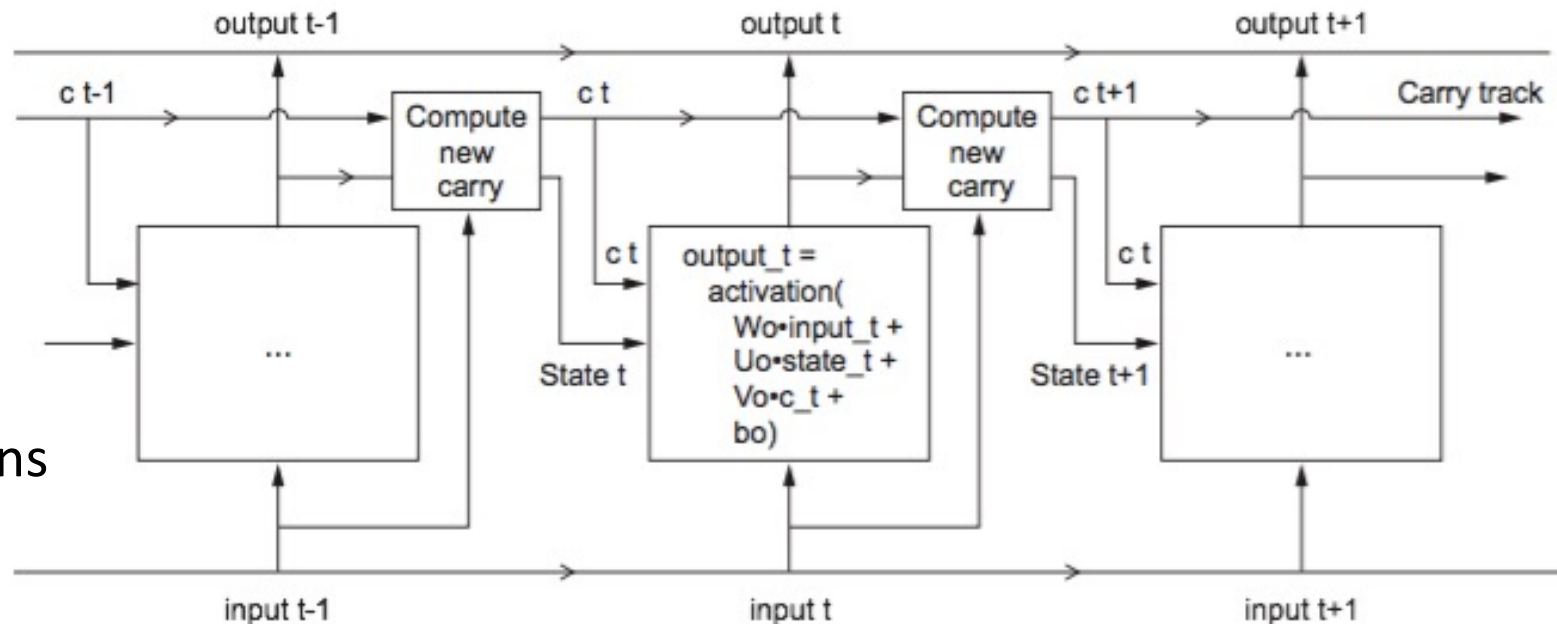
LSTMs

LSTM – **Long Short-Term Memory** – are recurrent networks that are characterized by maintaining "memory"

They have been introduced to solve the so-called problems of "vanishing gradients"

c_t – carry

Calculated
from 3
transformations

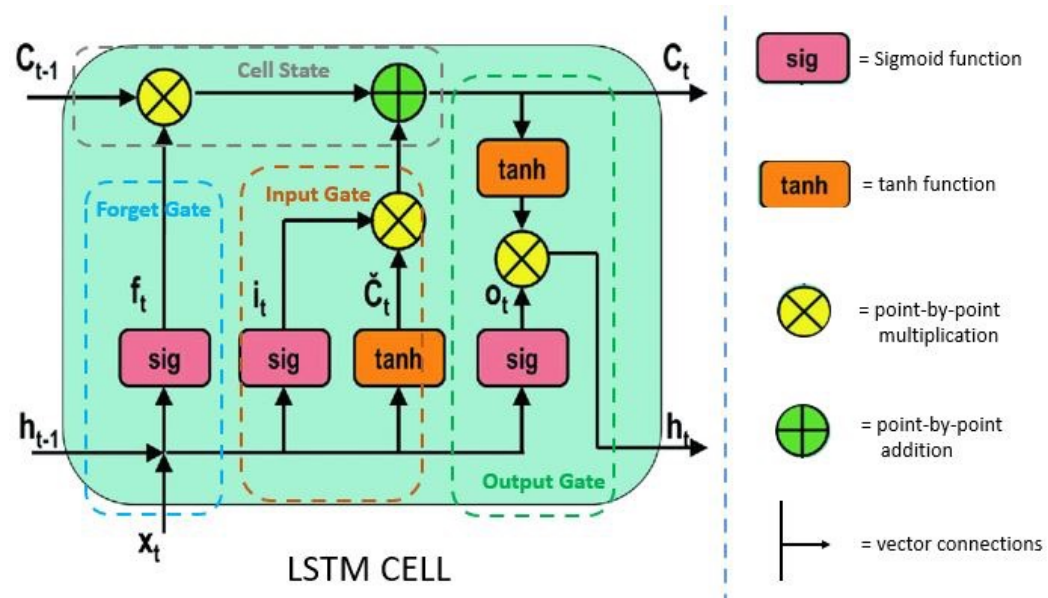


LSTMs - pseudocode

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(c_t, Vo) + bo)
i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wk) + bk)
```

```
c_{t+1} = i_t * k_t + c_t * f_t
```

Names of the gates (forget, input, ...) are merely indicative of their perceived “role”



Recurrent networks: supplementary notes

LSTM layers can be replaced by **GRU layers** with similar functionality (simpler – fewer weights to train)

GRU or LSTM layers can be **stacked** creating models with higher capacity (parameter `num_layers` in Pytorch)

You can consider using **Dropout** in these layers to address overfitting (check the examples in the python code - practice)

Bidirectional recurrent networks may be used in cases where both the natural order of the sequences and their reverse may make sense (flag defined in the code)